

Samuel “i9ine” O.
Backend Software Engineer
🌐 the-real-i9.github.io/i9ine



📞 +2348106545254
✉️ oluwarinolasam@gmail.com
🌐 linkedin.com/in/samueli9ine
🐙 github.com/the-real-i9

About Me.

Backend Engineer focused on building high-performance APIs, services, and systems involving design, architecture, and algorithmic problem-solving to solve real-world problems, while meeting both user and business demands. Experienced in building complex APIs, real-time communication systems, and data-intensive backend services using Go and Node.js. Skilled at translating the real-world into efficient data models, performance optimization, and designing clean paths through technical complexity.

Skills.

- + Go
- + JavaScript
- + TypeScript
- + Node.js
- + Express.js
- + PLpgSQL
- + C
- + PostgreSQL
- + Neo4j
- + Redis
- + SQL
- + CypherQL
- + HTTP
- + HTTP/3 (QUIC)
- + WebSocket
- + gRPC
- + REST API
- + Google Cloud Platform
- + Git & GitHub
- + GitHub Actions
- + Unit & API Testing
- + Postman
- + OpenAPI Spec.
- + AsyncAPI Spec.

Projects.

Social Media API - i9lyfe Started since Dec. 2023

A full-fledged social media backend API built in Go, featuring user authentication, content sharing, content discovery and user interaction, real-time notifications, user following, user chat, and scalable architecture using PostgreSQL, Redis, Neo4j, and GCS.

Technical highlights:

- I store JWT and session data in encrypted cookie for authentication and stateless session management, ensuring security and scalability.
- I employ the event-sourcing pattern; client request handlers queue events (e.g. users' reaction to post) into Redis streams, from which dedicated background workers dequeue and execute background tasks (e.g. incrementing reactions count on post in Redis cache, notifying post owners, performing expensive operations in the primary database). This allows client requests to spend the smallest, inevitable processing time, delivering fast user experience.
- I use Redis's sorted set data structure to serve cursor-based, paginated results (e.g. post comments, user chats, chat messages, notifications etc.) to the client. Each result item includes a cursor data that can be supplied on the next request for a new chunk of N items.
- Client requests for aggregate data (e.g. reactions count on post) are computed in constant time from Redis's set data structure using ZCard (sorted) or SCard (unsorted). No linear-time aggregate function is executed, reducing latency and enhancing scalability.
- I serve all READ requests from the cache; practically, whole data is built from parts in relevant cache entries. This offers fast user experience. Meanwhile, cached data are dynamically made fresh by relevant WRITE requests, zeroing the chances of having a stale cache at anytime. However, for certain data results, “eventual consistency” reasonably holds.

Visit Repo (<https://github.com/the-real-i9/i9lyfe-server>) for more technical highlights, architectural diagrams, and ER diagram.

Go

PostgreSQL

Redis (Streams, Pub/Sub, Queue [LPOP/RPUSH])

GCS

Skills. (continued)

Frontend Skills & Tools

- + HTML5
- + CSS3
- + TailwindCSS
- + Web APIs
- + React (JSX, TSX)
- + Vite
- + Figma

Soft Skills & Extra.

- + Logical Reasoning
- + Problem Solving
- + Product Thinking
- + UX Thinking
- + Communication
- + Graphics Design
- + Touch Typing (>60WPM)

Projects. (continued)

Chat App API - i9chat ————— Started since Feb. 2024

A full-fledged chat app API built in Go, featuring user authentication, direct and group messaging using text or voice, media sharing, group creation and management, media sharing, user search, and scalable architecture using Neo4j, Redis, and GCS.

Technical highlights:

- I switched from a relational database to a graph database because, most of my SQL database logic, especially those of group management, involved a combination of several READ and WRITE queries. Even after employing stored functions, each statement still does full index scan. A Neo4j query, on the other hand, is procedural by default, has the ability to retain pointer to nodes through intermediate clauses and perform its scan only within a branch of the graph; these proved Neo4j a more efficient alternative for my needs.
- Chat history is served from a Redis Sorted Set, while I use Redis Stream’s stream message ID for ordering, so that received messages appear in the order they were delivered rather than in the order they were created—the way WhatsApp works.
- I use an abstract “ChatEntry” entity to represent an item in the chat history, which may be a message, a reaction, or a group activity. While each of these are an entity on their own, the “ChatEntry” generalization allows for a more efficient processing of chat history READ requests, where we won’t have to UNION multiple entity READs and finally re-order them, a highly inefficient approach.
- All media processing and upload is offloaded to client-side. This eliminates a source of performance bottleneck, as many simultaneous user requests involving media processing will have to compete for server resources, an effect that will be felt even on user requests that don’t involve media processing unless additional infrastructure is added, which will incur unnecessary additional cost. The average modern device today has powerful media processing capabilities.

Visit Repo (<https://github.com/the-real-i9/i9chat-server>) for more technical highlights, architectural diagrams, and a Graph model overview.

Go

Neo4j

Redis (Streams, Pub/Sub, Queue [LPOP/RPUSH])

GCS

Remote File System API - i9rfs ————— Started since Apr. 2024

A remote file system API built in Node.js featuring user authentication, real-time file system operations, user-private directory tree and file system access, file metadata, and cloud object storage using WebSocket, Neo4j, and GCS.

Technical highlights:

- A graph data structure perfectly models a file system tree data structure; enough reason to choose a graph database over a relational database that’s not built for graph-like data structures and queries. I initially used PostgreSQL, where I wrote extremely complex, mind-boggling stored-function logic for each FS operation. This project introduced me to the world of graph databases.
- I utilize Neo4j’s variable expression matching extensively to confine object find tree traversal to the user’s root, and to process complex commands such as directory copy and delete which act recursively on sub-directories.

Visit Repo (<https://github.com/the-real-i9/i9rfs-server>) for more technical highlights, architectural diagrams, and a Graph model overview.

Node.js/Express.js

TypeScript

Neo4j

WebSocket

GCS